Spring Boot 3.5.6 개발 기본

inky4832@daum.net

7. JPA

- Spring Data JPA와 Hibernate
- Repository 패턴과 Query 메서드
- 고급 JPA 기능

7.1 Spring Data JPA와 Hibernate

JPA (Jakarta Persistence API)

- 개념
  - Java 객체(Object)와 관계형 데이터베이스(Relational Database)간의 매핑(Object Relational Database: ORM)을 **표준화한 인터페이스**
  - 즉 개발자가 SQL을 직접 작성하지 않고도 자바 객체 중심의 데이터베이스 프로그래밍을 가능하게 지원

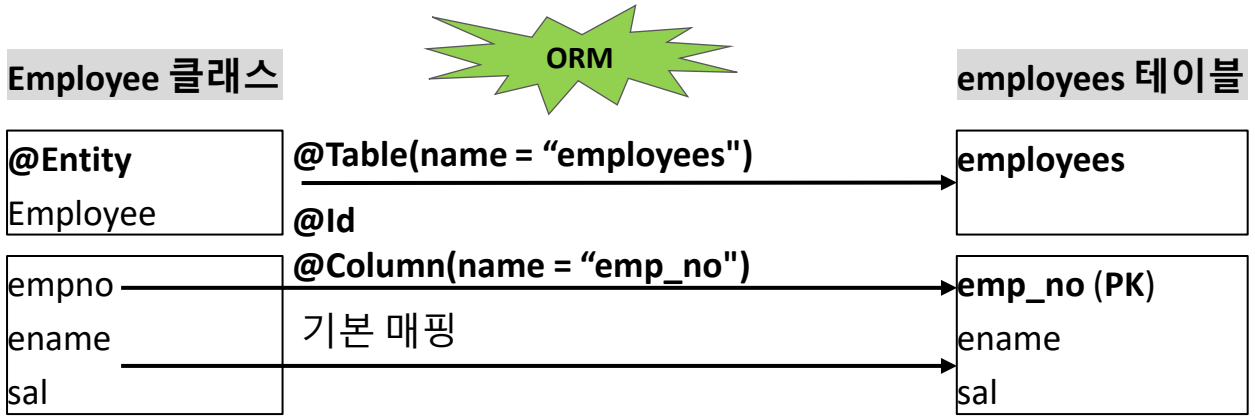
- 특징

항목	설명
객체지향적	객체 필드와 DB 컬럼을 매핑 시켜서 자바코드 중심으로 데이터 처리
SQL 의존성 감소	SQL 작성 없이 CRUD 수행 가능
표준화	Hibernate 등 여러 ORM 구현체를 JPA 표준 인터페이스로 통일
생산성/유지보수 향상	반복적인 코드제거, 트랜잭션 자동관리/ 비즈니스 로직 중심

7.1 Spring Data JPA와 Hibernate

ORM 개요

- 개념
  - Object-Relational Mapping (객체 관계 매핑)
  - 객체는 객체대로 설계하고 관계형 데이터베이스는 관계형 데이터베이스로 설계
  - ORM 프레임워크가 중간에서 매핑 처리



7.1 Spring Data JPA와 Hibernate

주요 구성 요소

구성 요소	설명
Entity	DB 테이블과 매핑되는 자바 클래스
EntityManager	Entity를 저장, 수정, 삭제, 조회하는 API
EntityManagerFactory	EntityManager 생성 관리
Persistence Context	영속성 컨텍스트 Entity 객체를 보관하고 상태를 추적하는 JPA의 내부 저장소
Persistence Unit	JPA 설정 정보가 저장된 논리 단위 ex. application.yml
JPQL	SQL과 유사하지만 Entity 객체를 대상으로 쿼리 수행

7.1 Spring Data JPA와 Hibernate

Entity (엔티티)

- 개념
  - DB 테이블과 1:1 로 매핑되는 자바 클래스 (@Entity 로 표시)
  - 식별자(PK)를 반드시 가져야 되며 JPA가 영속성 컨텍스트에서 상태를 추적함

주요 어노테이션	설명
@Entity	JPA가 관리할 엔티티 클래스 지정
@Table(name = "table_name")	매핑할 DB 테이블명 지정
@Id	기본 키(PK) 지정
@GeneratedValue(strategy = GenerationType.IDENTITY)	PK 자동 증가 전략 지정
@Column(name = "column_name")	컬럼 매핑 및 속성 지정
@OneToOne, @OneToMany, @ManyToOne, @ManyToMany	엔티티 간 연관관계 설정
@JoinColumn(name = "foreign_key")	외래키(FK) 매핑 설정

7.1 Spring Data JPA와 Hibernate

Entity (엔티티) 예시

```
@Entity
@Table(name = "users")
public class User {

  @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  @Column(name="username" , nullable = false, length = 50)
  private String name;

  @Column(nullable = false, unique = true, length = 120)
  private String email;

  protected User() {}
  public User(String name, String email) { this.name = name; this.email = email; }

  // getter/setter 생략
}
```

7.1 Spring Data JPA와 Hibernate

영속성 컨텍스트 (Persistence Context)

- 개념
  - 트랜잭션 동안 엔티티 객체를 1차 캐시에 보관하고 상태를 추적하는 JPA 내부 저장소

- 주요 기능

기능	설명
1차 캐시	같은 트랜잭션에서 같은 PK를 2번 조회해도 DB 재조회 없이 이미 읽은 엔티티를 재사용 (성능 향상 기대)
동일성 보장	같은 PK는 동일 객체로 유지
변경 감지	엔티티 필드가 바뀌면 트랜잭션 종료 시 자동으로 UPDATE 문 실행
쓰기 지연	여러 INSERT 작업을 트랜잭션 커밋 직전에 배치로 모아서 실행 가능
지연 로딩	연관관계 시 내용을 가져올 때 전체를 가져오지 않고 필요한 정보만 먼저 로딩 가능

7.1 Spring Data JPA와 Hibernate

엔티티 생명주기

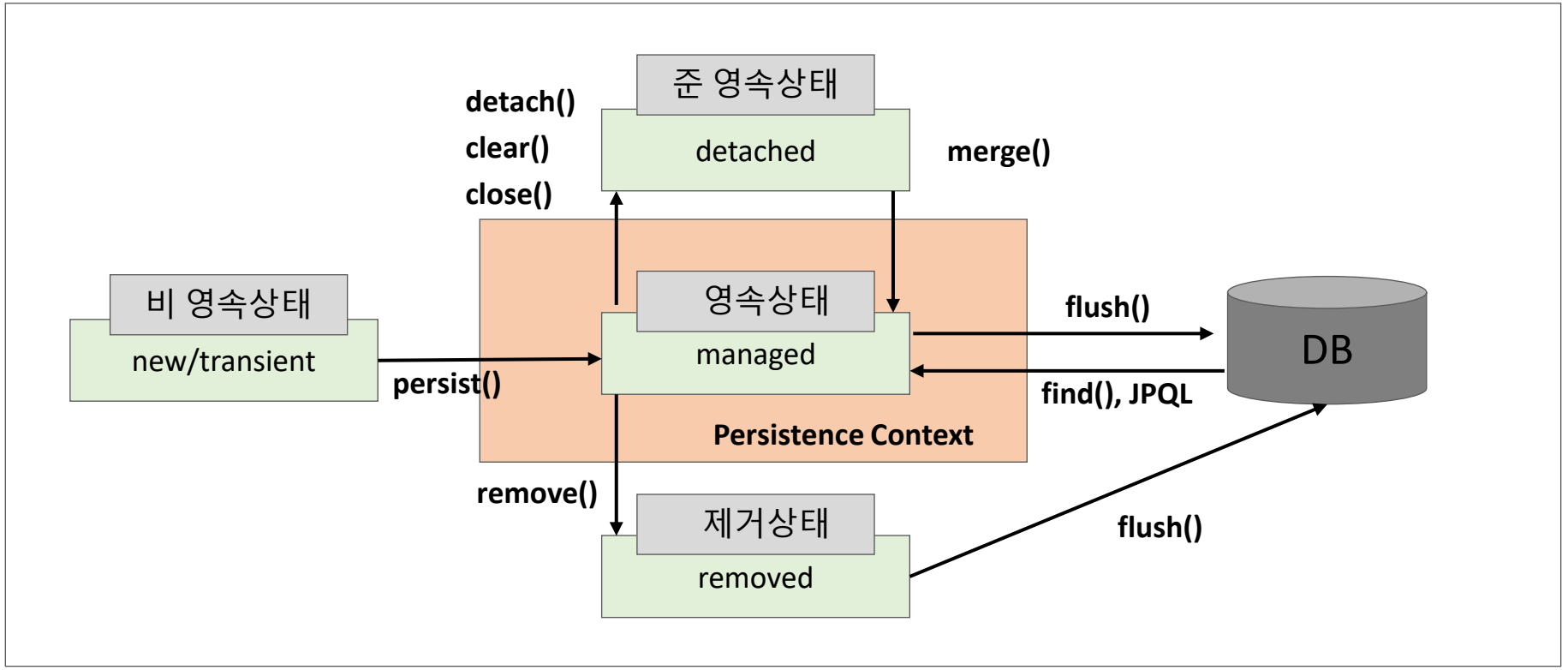
- 개념
  - 엔티티 생성부터 소멸까지의 라이프사이클
  - https://docs.hibernate.org/orm/6.4/userguide/html_single/#pc

- 4가지 상태

상태	설명
비영속 (new/transient)	JPA가 모르는 순수 자바 객체. 아직 영속성 컨텍스트에 등록 안됨
영속 (managed)	영속성 컨텍스트(1차 캐시)가 관리 중인 상태. 동일성보장, 변경 감지, 쓰기 지연 등 모든 이점 적용 가능
준영속 (detached)	한 때 영속이었으나 컨텍스트에서 분리된 상태. 더 이상 변경 감지/동일성 보장 적용 안됨
삭제 (removed)	삭제 예약 상태. 커밋 시 DELETE 실행

7.1 Spring Data JPA와 Hibernate

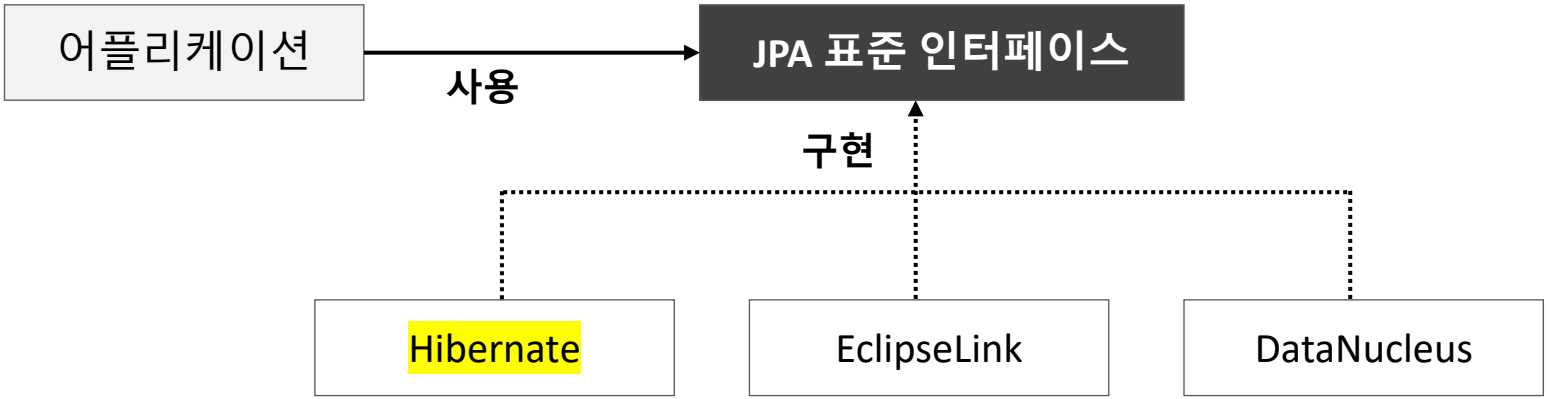
엔티티 생명주기 동작 흐름



7.1 Spring Data JPA와 Hibernate

Hibernate

- 개념
  - JPA 표준을 실제로 구현한 대표 ORM 프레임워크
  - 일반적으로 어플리케이션 코드는 JPA API를 사용하고, 런타임에 Hibernate가 실제 SQL문을 만들어 DB에 반영



https://docs.hibernate.org/orm/6.4/userguide/html_single/

7.1 Spring Data JPA와 Hibernate

Spring Data JPA

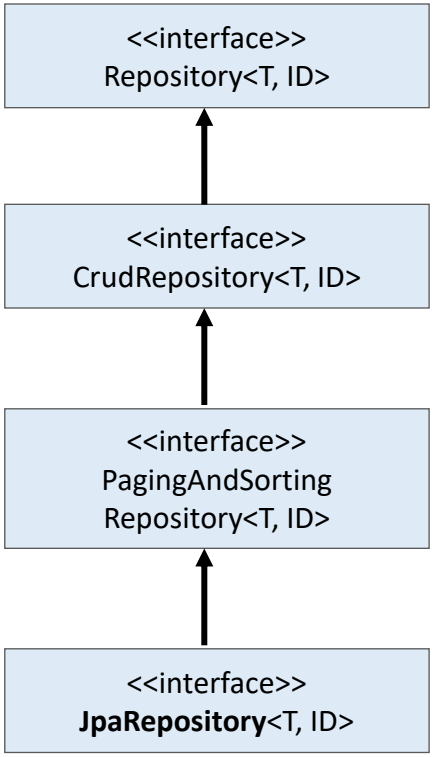
- 개념
  - JPA(Hibernate)를 더 쉽게 사용할 수 있도록 추상화한 Spring Framework의 하위 프로젝트
  - JPA를 기반으로 한 데이터 접근 계층(Data Access Layer)의 개발을 단순화함

- 특징

항목	설명
생산성 향상	Repository 인터페이스만 정의해도 CRUD 메서드 자동 구현
일관된 API	데이터베이스 종류와 상관없이 동일한 코드 사용
트랜잭션 통합	Spring의 @Transactional 과 완벽하게 통합
명시적 쿼리 지원	@Query 어노테이션으로 복잡한 쿼리 정의 가능
페이지/정렬 지원	Pageable, Sort 객체로 손쉬운 페이징 및 정렬 가능
Auditing 기능	엔티티 생성/수정 시 자동으로 시간/작성자 기록

7.1 Spring Data JPA와 Hibernate

Repository 인터페이스 구조



Marker Interface

Spring Data에 소속된 interface로
save, findById, finaAll, count, delete, existsById 등 기본 CRUD 기능이 선언

Spring Data에 소속된 interface로
Sort와 Pageable을 이용해서 정렬과 페이징을 처리하는 기능이 선언
ex. findAll(Pageable), findAll(Sort)

실제 사용할 interface 로서 상위 Repository 보다 훨씬
많은 기능이 포함됨 ex. JpaRepository**<엔티티, PK타입>**

7.1 Spring Data JPA와 Hibernate

JpaRepository 예시

```
public interface MemberRepository extends JpaRepository<Member, Long> {

    // CRUD 기능을 가진 메서드 자동 제공
      ex.  save , findById, findAll,  deleteById
}
```

```
@Service
public class MemberService {

   private MemberRepository repo;

   pubic MemberService(MemberRepository repo){
                this.repo = repo;
   }

   @Transactional
   public Member getMember(String id) {
      return repo.findById(id);
   }
}
```

7.1 Spring Data JPA와 Hibernate

의존성 설정

- 의존성
  - iimplementation 'org.springframework.boot:spring-boot-starter-data-jpa'

- 포함 라이브러리

라이브러리	설명
spring-data-jpa	JpaRepository, @Query 등 Spring Data 기능
spring-orm	EntityManagerFactory, 트랜잭션 매니저 구성
Jakarta-persistence-api	JPA 표준 인터페이스
org.hibernate.orm:hibernate-core	JPA 구현체 (기본 Hibernate)

7.1 Spring Data JPA와 Hibernate

개발 순서

단계	작업	설명
1 단계	Entity 생성	@Entity, @Id, @GeneratedValue 등으로 DB 테이블과 매핑 Lombok 라이브러리 이용
2 단계	Repository 인터페이스 정의	JpaRepository<엔티티, PK타입> 상속
3 단계	Service 계층 구현	Repository 주입 받아 비즈니즈 로직 작성
4 단계	Controller 작성	HTTP 요청 처리 및 서비스 호출
5 단계	application.yml 작성	DataSource, hibernate 설정, DDL 자동 생성 옵션 등 지정

7.1 Spring Data JPA와 Hibernate

application.yml 핵심 속성

속성	설명	권장값 / 예시 / 비고
spring.datasource.url spring.datasource.username spring.datasource.password spring.datasource.driver-class-name	DB 접속 URL DB 사용자 DB 비밀번호 드라이버	jdbc:mysql://localhost:3306/testdb root 1234 com.mysql.cj.jdbc.Driver
spring.datasource.hikari.pool-name spring.datasource.hikari.maximum-pool-size spring.datasource.hikari.idle-timeout spring.datasource.hikari.connection-timeout	커넥션풀 이름 최대 커넥션 유휴 커넥션 종료 대기 타임 아웃	모니터링/로그 구분에 도움 20 600000 (10분) 30000 (30초), 커넥션 획득 실패 시간
spring.jpa.open-in-view (OSIV)	영속성 컨텍스트 유지전략. true값 지정 시 요청전체구간 (Controller~View)에서 유효.	실무 권장 false (서비스 계층내에서만 유효)
spring.jpa.hibernate.ddl-auto	스키마 자동 생성	none,create,create-drop.update,validate
spring.jpa.properties.hibernate.format_sql logging.level.org.hibernate.SQL logging.level.org.hibernate.orm.jdbc.bind	SQL 포맷팅 실행 SQL 로그 출력 파라미터 값 출력	개발만 true Hibernate의 SQL 로그를 출력, ex. trace Hibernate 6 기준, ex. Trace
spring.jpa.properties.hibernate.dialect	방언	자동 추론되지만 명시도 가능 ex. org.hibernate.dialect.MySQLDialect

7.1 Spring Data JPA와 Hibernate

application.yml 핵심 속성 예시

```yaml
server:
  port: 8090

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/testdb
    username: root
    password: 1234
    #    driver-class-name: com.mysql.cj.jdbc.Driver
    hikari:
      pool-name: app-hikari
      maximum-pool-size: 20
      minimum-idle: 5
      idle-timeout: 600000            # 10분
      max-lifetime: 1800000           # 30분
      connection-timeout: 30000  # 30초

  thymeleaf:
    cache: false

  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate          # 운영 권장: 스키마 자동 변경 금지
    properties:
      hibernate:
        format_sql: true      # SQL 포맷팅

logging:
  level:
    root: info
    org.springframework.web: debug
    org.hibernate.SQL: trace                    # 실행되는 SQL
    org.hibernate.orm.jdbc.bind: trace     # 바인딩 파라미터
    org.hibernate.orm.jdbc.extract: trace  # 결과 매핑/추출
```

7.1 Spring Data JPA와 Hibernate

Lombok

- 개념
  - Java의 반복적인 코드를 자동으로 생성해주는 컴파일 타임 어노테이션 처리기
  - 개발자가 작성해야 될 보일러플레이트 코드 등을 자동으로 생성함

    ex. getter/setter, toString(), equals(), hashCode(), 생성자, Builder

- 특징

항목	설명
주요 사용처	DTO, Entity, VO, Record 대체용 클래스
주요 목적	반복적인 코드 자동 생성
사용 시점	컴파일 시점
대표 어노테이션	@Getter, @Setter, @ToString, @NoArgsConstructor, @AllArgsConstructor, @Builder 등

7.1 Spring Data JPA와 Hibernate

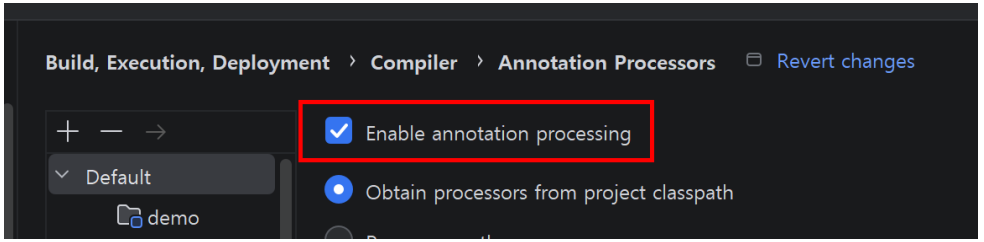
Lombok

- 의존성

```
dependencies {
    compileOnly 'org.projectlombok:lombok'
    annotationProcessor 'org.projectlombok:lombok'
    testCompileOnly 'org.projectlombok:lombok'
    testAnnotationProcessor 'org.projectlombok:lombok'
}
```

- Intellij IDE의 Lombok 활성화

settings > Build, Execution, Deployment > Compiler > Annotation Processors 에서 Enable annotation processing 체크 필요.



7.1 Spring Data JPA와 Hibernate

Lombok 주요 어노테이션

어노테이션	설명
@Getter / @Setter	모든 필드의 getter/setter 메서드 자동 생성
@ToString	객체 정보를 문자열로 표현하는 toString() 메서드 자동 생성
@EqualsAndHashCode	equals() 와 hashCode() 자동 생성
@NoArgsConstructor	매개변수 없는 기본 생성자 생성
@AllArgsConstructor	모든 필드를 매개변수로 받는 생성자 생성
@RequiredArgsConstructor	final 또는 @NotNull 필드만 포함한 생성자 생성
@Builder	객체를 빌더 패턴으로 생성 가능 컬렉션 필드에는 @Builder.Default 지정하여 자동으로 초기화 지원
@Data	@Getter, @Setter,@ToString,@EqualsAndHashCode, @RequiredArgsConstructor 포함 (DTO 적합, Entity 지양)
@Slf4j	로깅용 Logger 자동생성

7.1 Spring Data JPA와 Hibernate

Lombok 예시

```
import lombok.*;

@Data // @Getter + @Setter + @ToString + @EqualsAndHashCode + @RequiredArgsConstructor
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class MemberDTO {

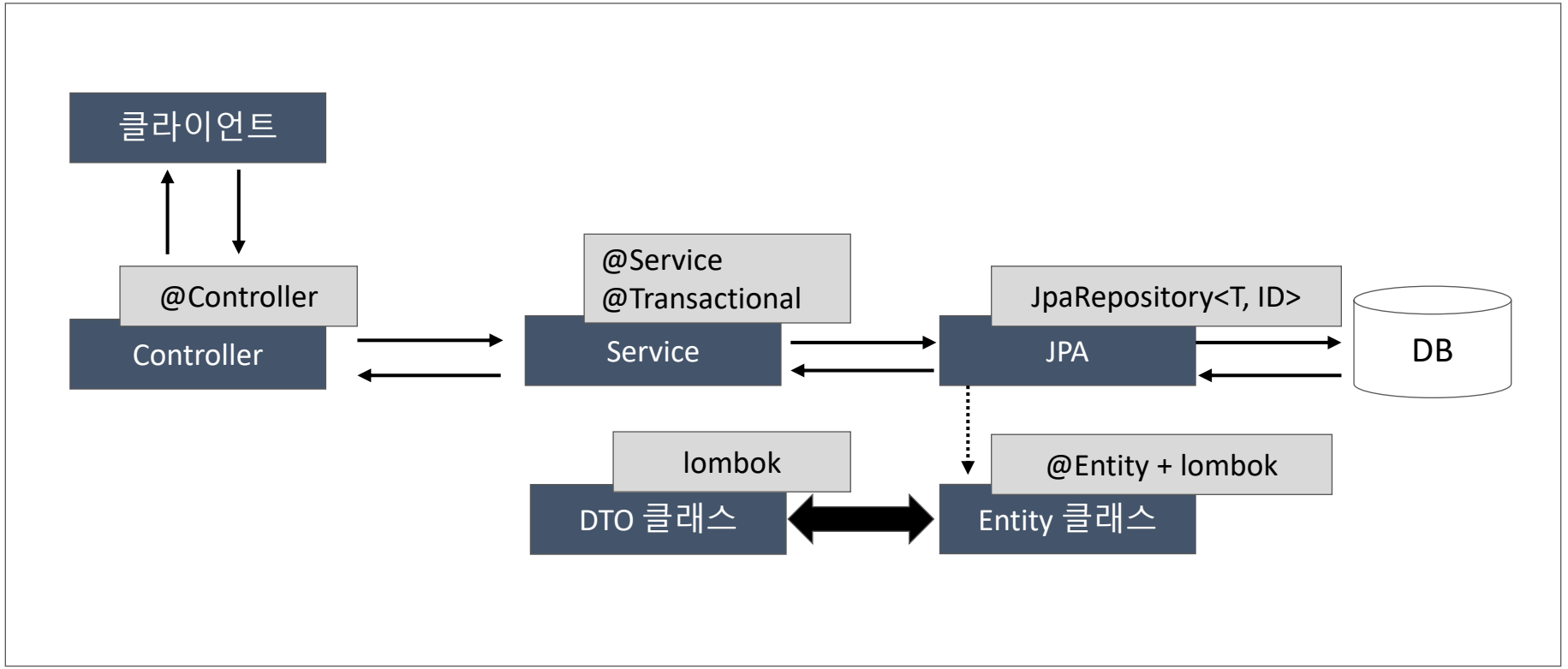
    private Long id;
    private String name;
    private String email;
    private int age;

    @Builder.Default
    private List<Employee> employees = new ArrayList<>();


}
```

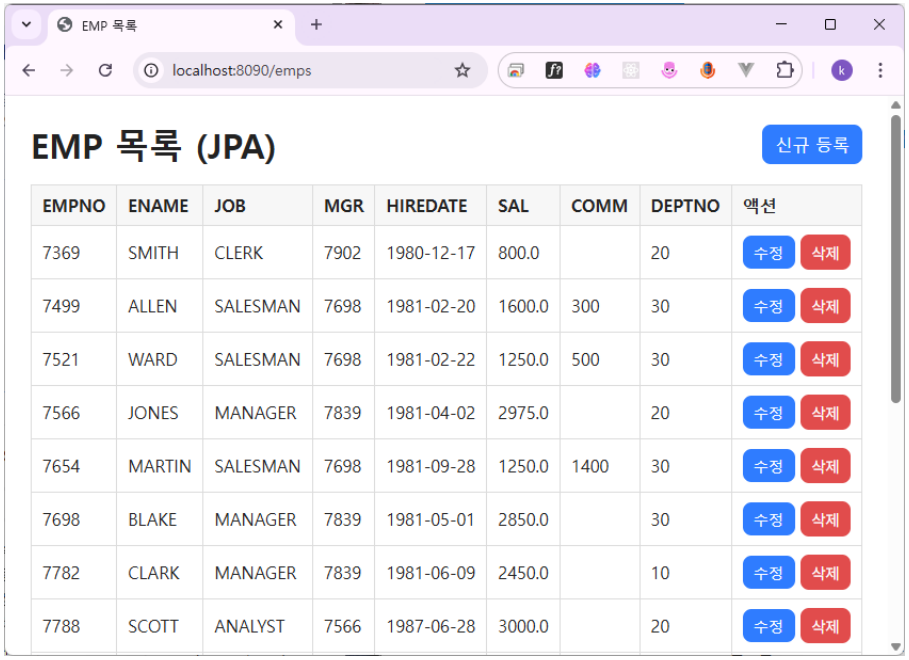
7.1 Spring Data JPA와 Hibernate

JPA 연동 아키텍처



7.1 Spring Data JPA와 Hibernate

실습 (JPA 기본)



7.1 Spring Data JPA와 Hibernate

실습 (JPA 기본)

```
# application.yml
server:
  port: 8080

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/testdb
    username: root
    password: 1234
    hikari:
      pool-name: app-hikari
      maximum-pool-size: 20
      minimum-idle: 5
      idle-timeout: 600000              # 10분
      max-lifetime: 1800000             # 30분
      connection-timeout: 30000         # 30초
  thymeleaf:
    cache: false

  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate        # 운영 권장: 스키마 자동 변경 금지
    properties:
      hibernate:
        format_sql: true                # SQL 포맷팅

logging:
  level:
    root: info
    org.springframework.web: debug
    org.hibernate.SQL: trace                # 실행되는 SQL
    org.hibernate.orm.jdbc.bind: trace      # 바인딩 파라미터
    org.hibernate.orm.jdbc.extract: trace   # 결과 매핑/추출
```

7.1 Spring Data JPA와 Hibernate

실습 (JPA 기본)

// EmpDTO.java

```
package com.example.demo.dto;

import java.time.LocalDate;
import lombok.*;

@Getter @Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class EmpDTO {
    private Integer empno;
    private String ename;
    private String job;
    private Integer mgr;
    private LocalDate hiredate;
    private Double sal;
    private Integer comm;
    private Integer deptno;

}
```



// Emp.java

```
package com.example.demo.entity;

import java.time.LocalDate;
import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "EMP")
@Getter @Setter @NoArgsConstructor
@AllArgsConstructor
@Builder
public class Emp {

    @Id
    @Column(name = "EMPNO")
    private Integer empno;
    private String ename;
    private String job;
    private Integer mgr;
    private LocalDate hiredate;
    private Double sal;
    private Integer comm;
    private Integer deptno;
}
```

7.1 Spring Data JPA와 Hibernate

실습 (JPA 기본)

// EmpRepository.java

```
package com.example.demo.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.demo.entity.Emp;


public interface EmpRepository extends JpaRepository<Emp,
Integer> {

    // findAll(), findById(), save(), deleteById 기본 제공

}
```

// EmpServiceImpl.java

```
package com.example.demo.service;

…

@Service
@RequiredArgsConstructor
public class EmpServiceImpl implements EmpService {

    private final EmpRepository empRepository;

@Override
    public List<EmpDTO> findAll() {
        return empRepository.findAll().stream()
                .map(this::fromEmpToEmpDTO)
                .toList();
    }
    @Override
    public Optional<EmpDTO> findById(Integer empno) {
        return
empRepository.findById(empno).map(this::fromEmpToEmpDTO);
    }
    // ===== 저장(신규) =====
    @Override
    @Transactional
    public EmpDTO save(EmpDTO dto) {
        Emp entity = fromEmpDTOToEmp(dto);
        Emp saved = empRepository.save(entity);
        return fromEmpToEmpDTO(saved);
    }
```

7.1 Spring Data JPA와 Hibernate

실습 (JPA 기본)

```
// ===== 수정(기존) =====
    @Override
    @Transactional
    public EmpDTO update(Integer empno, EmpDTO dto) {
        Emp entity = empRepository.findById(empno)
                .orElseThrow(
                () -> new EntityNotFoundException("Not found: " + empno));

        dto.setEmpno(entity.getEmpno());  //엔티티를 수정
        entity = fromEmpDTOToEmp(dto);

        Emp saved = empRepository.save(entity);
        return fromEmpToEmpDTO(saved);
    }

    @Override
    @Transactional
    public void delete(Integer empno) {
        if (!empRepository.existsById(empno)) return;
        empRepository.deleteById(empno);
    }
```

```
// ====== 내부 공통 메서드 ======
public EmpDTO fromEmpToEmpDTO(Emp entity) {
        EmpDTO dto = EmpDTO.builder()
                .empno(entity.getEmpno()).ename(entity.getEname())
                .job(entity.getJob()).mgr(entity.getMgr())
                .hiredate(entity.getHiredate()).comm(entity.getComm())
                .sal(entity.getSal()).deptno(entity.getDeptno())
                .build();
        return dto;
    }
public Emp fromEmpDTOToEmp(EmpDTO dto) {
        Emp entity = Emp.builder()
                .empno(dto.getEmpno())
                .ename(dto.getEname()).job(dto.getJob())
                .mgr(dto.getMgr()).hiredate(dto.getHiredate())
                .comm(dto.getComm()).sal(dto.getSal())
                .deptno(dto.getDeptno())
                .build();
        return entity;
    }
}
```

7.1 Spring Data JPA와 Hibernate

실습 (JPA 기본)

// EmpController.java

```
package com.example.demo.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

import com.example.demo.dto.EmpDTO;
import com.example.demo.service.EmpService;

import lombok.RequiredArgsConstructor;

@Controller
@RequiredArgsConstructor
@RequestMapping("/emps")
public class EmpController {

  private final EmpService service;

@GetMapping
  public String list(Model model) {
    model.addAttribute("emps", service.findAll());
    return "emp/list";
  }

  @GetMapping("/new")
  public String createForm(Model model) {
    model.addAttribute("emp", new EmpDTO());
    model.addAttribute("mode", "create");
    return "emp/add";
  }

  @PostMapping("/add")
  public String create(@ModelAttribute EmpDTO emp) {
     service.save(emp);
    return "redirect:/emps";
  }
```

7.1 Spring Data JPA와 Hibernate

실습 (JPA 기본)

```
@GetMapping("/{empno}/edit")
    public String editForm(@PathVariable("empno") Integer empno, Model model) {
        var emp = service.findById(empno).orElseThrow(() -> new IllegalArgumentException("Not found: " + empno));
        model.addAttribute("emp", emp);
        model.addAttribute("mode", "edit");
        return "emp/edit";
    }

    @PostMapping("/{empno}/update")
    public String update(@PathVariable("empno") Integer empno, @ModelAttribute EmpDTO emp) {
         service.update(empno, emp);
        return "redirect:/emps";
    }

    @PostMapping("/{empno}/delete")
    public String delete(@PathVariable("empno") Integer empno) {
        service.delete(empno);
        return "redirect:/emps";
    }
}
```

7.2 Repository 패턴과 Query 메서드

Repository 패턴

- 개념
  - 데이터 접근 로직(DAO, Data Access Object)을 캡슐화해서 비즈니즈 로직(Service)과 분리하는 설계 패턴
  - 데이터 소스(MySQL 등)에 직접 접근하지 않고 인터페이스 기반 추상화 계층을 통해 CRUD 기능을 표준화 함

항목	설명
단일 책임 원칙	Repository는 오직 데이터 접근만 담당한다.
유지보수 용이	데이터 저장소(DB)가 바뀌어도 Repository 내부만 수정하면 된다.
의존성 분리	Service는 데이터 처리 기술(JPA, MyBatis, JDBC 등)에 독립적이다.
테스트 용이성	Mock 이용해서 단위 테스트를 빠르고 쉽게 수행할 수 있다.

7.2 Repository 패턴과 Query 메서드

Repository 패턴

- 책임과 비책임
  - **책임 : 엔티티 저장/조회/삭제, 조건 검색, 페이징/정렬 등**
  - 비책임 : 복잡한 도메인 계산/규칙, 외부 시스템 연동, 트랜잭션 경계 설정 등

- DAO 차이점

항목	DAO	Repository
기준(중심)	테이블/SQL 중심	도메인/엔티티 중심
추상화 수준	SQL 문장 단위	도메인 개념(집합) 단위
인터페이스	작업(삽입/수정/삭제) 위주	언어적/업무적 의도가 드러나는 메서드
응답 모델	Row/DTO	엔티티/도메인 객체 중심

7.2 Repository 패턴과 Query 메서드

Repository 패턴

▪ 핵심 역할

요소	설명
Entity	데이터베이스 테이블과 1:1 매핑되는 클래스 ex. Product, Member, Order 등
Repository	엔티티의 CRUD 기능을 제공하는 인터페이스 JpaRepository<T, ID> 상속 시 기본 CRUD 메서드 자동 제공됨 ex. findById, findAll, save
Service	비즈니스 로직을 담당하고 트랜잭션 관리 수행
Controller	사용자 요청을 받아 서비스 호출 후 결과 반환

7.2 Repository 패턴과 Query 메서드

Query 메서드

- 개념
  - 쿼리를 직접 작성하지 않고 메서드 이름 규칙만으로 SQL를 자동 생성함

- 공식 사이트
  - **https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html#jpa.query-methods.query-creation**

Table 1. Supported keywords inside method names

Keyword	Sample	JPQL snippet
`Distinct`	`findDistinctByLastnameAndFirstname`	`select distinct … where x.lastname = ?1 and x.firstname = ?2`
`And`	`findByLastnameAndFirstname`	`… where x.lastname = ?1 and x.firstname = ?2`
`Or`	`findByLastnameOrFirstname`	`… where x.lastname = ?1 or x.firstname = ?2`
`Is, Equals`	`findByFirstname, findByFirstnameIs, findByFirstnameEquals`	`… where x.firstname = ?1` (or `… where x.firstname IS NULL` if the argument is `null`)

7.2 Repository 패턴과 Query 메서드

Query 메서드

- 기본 구조
  - **리턴타입 + find + By + [엔티티필드명] + [조건키워드]**
    ex. findByName, findBySalBetween, findByCommIsNull

- 주요 조건 키워드

항목	DAO	Repository
비교	LessThan, GreaterThan,Between	숫자/날짜 범위 검색
문자열	Like, Containing, StartingWith, EndingWith	문자열 패턴 매칭
논리값	True, False	Boolean 필드 조건
조합	And, Or, OrderBy	복합 조건 및 정렬
존재여부	ExistsBy, CountBy, DeleteBy	특정 조건 존재/삭제

7.2 Repository 패턴과 Query 메서드

Query 메서드 예시표

메서드명	의미	JPQL 예시
findByEname(ename)	특정 이름(ename)의 사원 조회	SELECT e FROM Emp e WHERE e.ename = :ename
findBySalBetween(min, max)	급여(sal)가 특정 범위에 포함된 사원 조회	SELECT e FROM Emp e WHERE e.sal BETWEEN :min AND :max
findByEnameContaining(keyword)	이름에 특정 문자열이 포함된 사원 조회	SELECT e FROM Emp e WHERE e.ename LIKE %:keyword%
findByCommIsNull()	커미션(comm)이 없는 사원 조회	SELECT e FROM Emp e WHERE e.comm IS NULL
findByJobAndDeptno(job, deptno)	특정 직무이면서 특정 부서에 속한 사원 조회	SELECT e FROM Emp e WHERE e.job = :job AND e.deptno = :deptno
findByHiredateAfter(date)	특정 날짜 이후에 입사한 사원 조회	SELECT e FROM Emp e WHERE e.hiredate > :date
findByOrderBySalDesc()	급여(sal) 기준으로 내림차순 정렬해서 조회	SELECT e FROM Emp e ORDER BY e.sal DESC
countByDeptno(deptno)	특정 부서의 사원수 조회	SELECT COUNT(e) FROM Emp e WHERE e.deptno = :deptno

7.2 Repository 패턴과 Query 메서드

Paging 과 Sorting 개념

- 페이징(Paging)
  - 데이터가 많을 때 한번에 모든 데이터를 보여주지 않고 한 페이지 당 일정 개수 씩 나주어 보여주는 기술

    ex. Page 1 (1~10번) Page 2(11~20번) …

- 정렬(Sorting)
  - 데이터를 가격순/날짜순 등, 정렬 기준에 따라 오름차순/내림차순으로 보여주는 기술

7.2 Repository 패턴과 Query 메서드

Paging 과 Sorting API

API	설명
Pageable	요청한 페이지 정보 (페이지 번호, 크기, 정렬 조건)를 담은 인터페이스
PageRequest	Pageable의 구현체로서 페이지 요청 객체를 생성할 때 사용 PageRequest.of(page, size, sort) 로 생성 ex. PageRequest.of(0, 10, Sort.by("price").descending())
Page<T>	조회 결과를 담은 객체. 추가로 전체 페이지 수, 현재 페이지 번호 등이 포함
Sort	데이터를 정렬 기준에 따라 오름차순/내림차순으로 정렬하는 클래스 Pageable과 거의 같이 사용됨. ex. Sort sort = Sort.by("price").descending();

7.2 Repository 패턴과 Query 메서드

Pageable API

메서드	설명
int getPageNumber()	현재 페이지 번호. 0 부터 시작
int getPageSize()	한 페이지당 데이터 개수 ex. 10
long getOffset()	조회 시작 위치 (offset) ex. Page 0은 0, Page 1은 10
Pageable next()	다음 페이지 요청 객체 반환
Pageable previousOrFirst()	이전 페이지 또는 첫 페이지 반환
boolean hasPrevious()	이전 페이지 존재 여부 확인
Sort getSort()	정렬 정보 반환

7.2 Repository 패턴과 Query 메서드

PageRequest API

메서드	설명
PageRequest.of(int page, int size)	기본 페이지 요청. 정렬 없음
PageRequest.of(int page, int size, Sort sort)	정렬 포함 페이지 요청 ex. PageRequest.of(0, 10, Sort.by("price").descending())
int getPageNumber()	현재 페이지 번호. 0 부터 시작
int getPageSize()	한 페이지당 데이터 개수 ex. 10
long getOffset()	조회 시작 위치 (offset) ex. Page 0은 0, Page 1은 10

7.2 Repository 패턴과 Query 메서드

Page<T> API

메서드	설명
List<T> getContent()	현재 페이지의 데이터 목록
int getNumber()	현재 페이지 번호. 0부터 시작
int getSize()	페이지당 데이터 개수
int getNumberOfElements()	현재 페이지 내 데이터 수
long getTotalElements()	전체 데이터 개수
int getTotalPages()	전체 페이지 수
boolean hasNext() / hasPrevious()	다음 / 이전 페이지 존재 여부
boolean isFirst() / isLast()	현재 페이지가 첫 페이지/마지막 페이지 여부
Pageable getPageable()	현재 페이지의 Pageable 정보 반환
Page<T> map(Function<T,R> converter)	페이지 내용 매핑(DTO 변환 등)

7.2 Repository 패턴과 Query 메서드

Sort API

메서드	설명	예시
Sort.by(String… properties)	기본 오름차순 정렬	Sort.by("price")
Sort.by(Sort.Order... orders)	다중 정렬	Sort.by(Sort.Order.asc("name"), Sort.Order.desc("price"))
Sort.asc(String property)	특정 필드 오름차순 정렬	Sort.asc("name")
Sort.desc(String property)	특정 필드 내림차순 정렬	Sort.desc("price")
sort.ascending()	현재 Sort를 오름차순 전환	sort.ascending()
sort.descending()	현재 Sort를 내림차순 전환	sort.descending()
sort.and(Sort other)	기존 정렬에 추가 정렬	Sort.by("deptno").and(Sort.by("sal").descending())
Sort.Order.getProperty()	정렬 기준 필드명 반환	-

7.2 Repository 패턴과 Query 메서드

Paging & Sorting 예시

ex.

```
Pageable pageable = PageRequest.of(0, 10, Sort.by("sal").descending());
Page<Emp> page = empRepository.findByJob("SALESMAN", pageable);
```

ex.

```
Pageable pageable = PageRequest.of(0, 10, Sort.by("price").descending());
Page<Product> page = productRepository.findByCategory("Book", pageable);

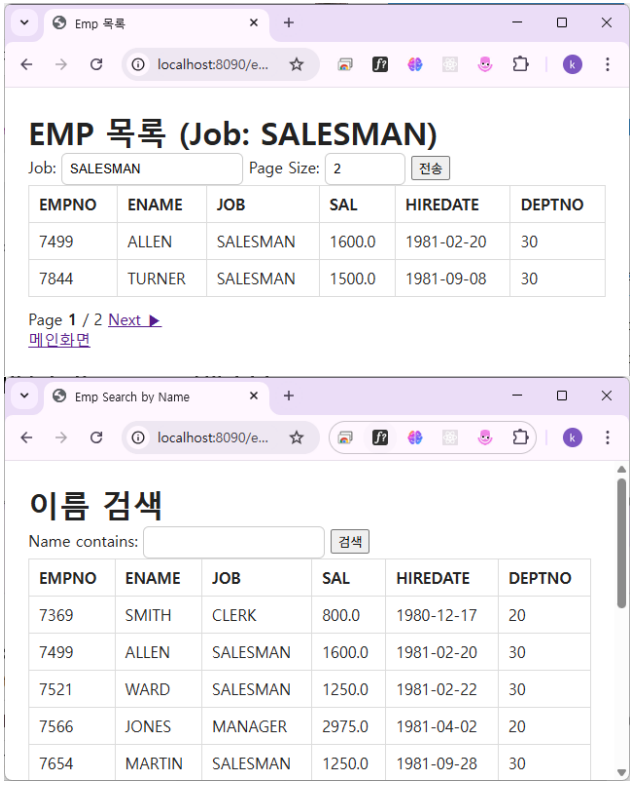
System.out.println("총 페이지 수: " + page.getTotalPages());
System.out.println("전체 데이터 수: " + page.getTotalElements());
System.out.println("현재 페이지 번호: " + page.getNumber());
page.getContent().forEach(p -> System.out.println(p.getName()));
```

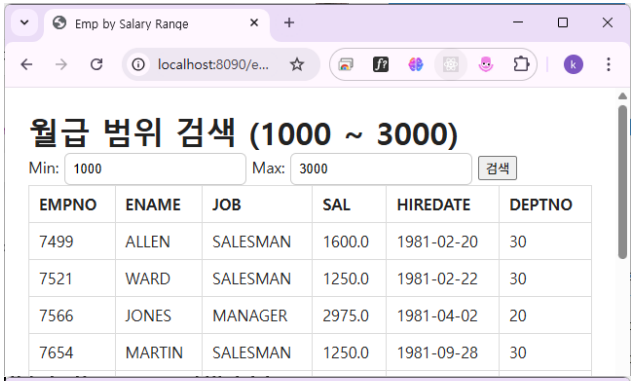
ex.

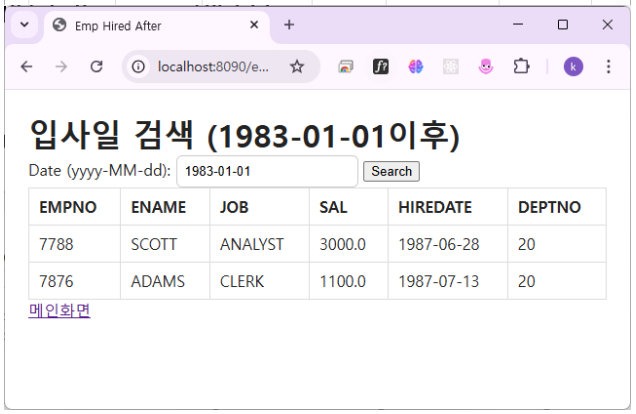
```
Sort sort = Sort.by(
  Sort.Order.asc("deptno"),
  Sort.Order.desc("sal")
);
Pageable pageable = PageRequest.of(0, 10, sort);
Page<Emp> page = empRepository.findAll(pageable);
```

7.2 Repository 패턴과 Query 메서드

실습 (Query 메서드+ Pageable)







7.2 Repository 패턴과 Query 메서드

실습 (Query 메서드+ Pageable)

```
// EmpRepository.java
package com.example.demo.repository;

…

public interface EmpRepository extends JpaRepository<Emp,
Integer> {
    // 단일 조건
   List<Emp> findByEname(String ename);
   List<Emp> findByJob(String job);

   // 부분 문자열 검색
   List<Emp> findByEnameContaining(String keyword);

// 범위 조건
   List<Emp> findBySalBetween(BigDecimal min, BigDecimal max);

   // 날짜 조건
   List<Emp> findByHiredateAfter(LocalDate date);
   List<Emp> findByHiredateBefore(LocalDate date);

   // 정렬/페이징
   Page<Emp> findByJob(String job, Pageable pageable);

   // 집계/존재여부
   long countByDeptno(Integer deptno);
   boolean existsByEname(String ename);

   // 삭제
   long deleteByDeptno(Integer deptno);
}
```

```
// EmpController.java
@GetMapping("/list")
public String list(@RequestParam(value = "job", defaultValue = "SALESMAN")
String job,
                   @RequestParam(value = "page",defaultValue = "0") int page,
                   @RequestParam(value = "size",defaultValue = "2") int size,
                   Model model) {
    Page<EmpDTO> result = service.listByJob(job, page, size);
    model.addAttribute("page", result);
    model.addAttribute("job", job);
    return "emp/list";
}
```

```
// EmpServiceImp.java
public Page<EmpDTO> listByJob(String job, int page, int size) {
    Pageable pageable = PageRequest.of(page, size, Sort.by("sal").descending());
    Page<Emp> repo = empRepository.findByJob(job, pageable);
    return repo.map(this::fromEmpToEmpDTO);
}
```

7.2 Repository 패턴과 Query 메서드

실습 (Query 메서드+ Pageable)

```
<!-- list.html -->
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
 <head>
  <meta charset="UTF-8" />
  <title>Emp 목록</title>
  <link rel="stylesheet" th:href="@{/css/styles.css}" />
 </head>
 <body>
  <h1>EMP 목록 (Job: <span th:text="${job}"></span>)</h1>
  <form method="get" th:action="@{/emps/list}">
   Job: <input name="job" th:value="${job}" /> Page Size:
   <input
    type="number"
    name="size"
    th:value="${page.size}"
    min="1"
    max="100"
   />
   <button type="submit">전송</button>
  </form>
```

```
<table>
   <tr>
    <th>EMPNO</th>
    <th>ENAME</th>
    <th>JOB</th>
    <th>SAL</th>
    <th>HIREDATE</th>
    <th>DEPTNO</th>
   </tr>
   <tr th:each="e : ${page.content}">
    <td th:text="${e.empno}"></td>
    <td th:text="${e.ename}"></td>
    <td th:text="${e.job}"></td>
    <td th:text="${e.sal}"></td>
    <td th:text="${e.hiredate}"></td>
    <td th:text="${e.deptno}"></td>
   </tr>
   <tr th:if="${#lists.isEmpty(page.content)}">
    <td colspan="6" style="text-align: center">No data</td>
   </tr>
  </table>
```

7.2 Repository 패턴과 Query 메서드

실습 (Query 메서드+ Pageable)

```
<div style="margin-top: 12px">
    <a
     th:if="${page.hasPrevious()}"
     th:href="@{|/emps/list?page=${page.number-1}&size=${page.size}&job=${job}|}"
     > Prev</a
    >
    <span>
     Page <b th:text="${page.number + 1}"></b> /
     <span th:text="${page.totalPages}"></span>
    </span>
    <a
     th:if="${page.hasNext()}"
     th:href="@{|/emps/list?page=${page.number+1}&size=${page.size}&job=${job}|}"
     >Next </a
    >
   </div>
   <a th:href="@{/emps}">메인화면</a>
  </body>
</html>
```

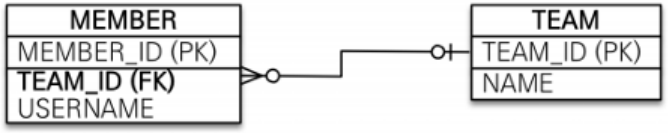
7.3 고급 JPA 기능

연관관계

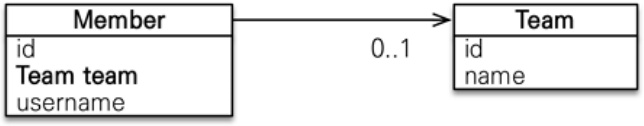
- 개념
  - 객체 간의 관계(참조)를 데이터베이스 테이블 간의 관계(외래키)로 매핑하는 것
  - 관계형 DB는 외래키(Foreign Key)를 통해 연관성을 표현하지만
  - 객체지향 프로그래밍에서는 참조(reference)를 통해 연관성을 표현함
  - JPA는 이러한 두 영역의 차이를 매핑(Mapping)으로 해결함

    ex. Member와 Team 관계 (N:1)

테이블 연관관계(FK 이용)



객체 연관관계(참조 이용)



7.3 고급 JPA 기능

연관관계 종류

관계 유형	설명	어노테이션	예시
1:1 (OneToOne)	한 엔티티가 다른 하나의 엔티티와 연관됨	@OneToOne	사원:사원상세정보
1:N (OneToMany)	하나의 엔티티가 여러 엔티티와 연관됨	@OneToMany	부서:사원
N:1 (ManyToOne)	여러 엔티티가 하나의 엔티티와 연관됨	@ManyToOne	**사원:부서**
N:M (ManyToMany)	다수가 다수와 연관됨	@ManyToMany	사원:프로젝트

7.3 고급 JPA 기능

연관관계 주요 어노테이션

어노테이션	적용위치	주요속성	설명
@ManyToOne	N:1 관계의 N 엔티티	fetch, cascade, optional	• N:1 관계 매핑, 외래키 소유자 ex. @ManyToOne(fetch=FetchType.LAZY)
@OneToMany	1:N 관계의 1 엔티티	fetch,cascade,mappedBy, orphanRemoval	• 주로 양방향에서 사용. ex. @OneToMany(mappedBy = “member", cascade = CascadeType.ALL)
@OneToOne	1:1 관계의 FK 엔티티	fetch, cascade, optional, mappedBy	• FK가 있는 엔티티가 owner ex. @OneToOne(fetch = FetchType.LAZY)
@ManyToMany	N:M 관계 양쪽	fetch, cascade, mappedBy	• 실문에서 사용 거의 안함 • 중간 엔티티 이용 권장
@JoinColum	외래키(FK) 소유 필드	name, nullable, unique, insertable, updatable	• 연관관계 owner 쪽의 FK컬럼지정 ex. @JoinColumn(name = "team_id")

7.3 고급 JPA 기능

테이블 외래키 기반의 매핑

- 개념
  - 관계형 데이터베이스는 외래키(Foreign Key)로 관계를 관리
  - JPA는 객체간 연관관계 설정시 @JoinColumn 어노테이션을 통해서 객체 참조를 DB 외래키 컬럼에 매핑함

- 내부 동작
  - ex. Member ---> Team, N:1

    @ManyToOne + @JoinColumn(name="team_id", nullable=false)

    이로서 객체 간 참조와 테이블 외래키(FK)가 일치하게 됨

- 주의사항

  외래키(FK)는 항상 N쪽 테이블에 존재함 ex. Member ------->Team, N:1

  조인 컬럼명은 DB 설계와 일치시켜야 유지보수에 유리

7.3 고급 JPA 기능

연관관계 방향(Direction)

- 개념
  - 객체의 참조는 단방향(one-way) 또는 양방향(Bidirectional)으로 구성 가능
  - DB에는 항상 단방향이지만 객체에서는 양쪽 탐색이 가능하도록 설계 가능
- 내부 동작
  - 단방향: Member---------->Team, N:1
  - 양방향: Member---------->Team + Team ---------> List<Member>
- 실무팁
  - 기본은 단방향으로 설계
  - 양방향은 조회 시 탐색 편의성이 필요한 경우에만 추가 설정
  - toString(), equals(), hashCode()에 양방향 필드를 넣으면 순환 참조로 StackOverflow 발생됨

7.3 고급 JPA 기능

연관관계 주인(Owner)

- 개념
  - 두 엔티티가 서로를 참조할 때, 실제 DB 외래키(FK)를 관리하는 쪽이 연관관계의 주인(owner)임
  - 주인만이 외래키 값을 변경할 수 있으며 비주인은 단순히 읽기만 가능함

- 내부 동작
  - @ManyToOne 쪽이 항상 주인(owner) ex. Member
  - 양방향 관계일 경우에는 @OneToMany(mappedBy="team") 설정하여 mappedBy 속성을 가진 쪽이 비주인임을 명시함 ex. Team

7.3 고급 JPA 기능

지연(Lazy) vs 즉시(Eager) 로딩

- 개념
  - JPA는 연관 엔티티를 언제 불러올지 결정하는 로딩 전략(fetch strategy)을 제공함
- 내부 동작
  - LAZY: 실제 사용시 DB 접근 조회, N+1 문제 발생 가능
  - EAGER: 즉시 JOIN SQL 수행하여 함께 조회 (기본 동작)
- 실무팁
  - 기본은 모든 연관관계 LAZY 설정
  - EAGER는 성능 저하 원인이 되므로 사용 지양

7.3 고급 JPA 기능

N+1 문제

- 개념
  - 하나의 데이터를 조회하기 위해 SQL을 1번 실행했는데, 그 데이터와 연관된 데이터를 가져오는 과정에서 추가 SQL이 N 번 실행되는 문제임

1. 부서 정보 조회

```
SELECT *
FROM DEPT;
```

DEPTNO	DNAME
10	개발부
20	영업부
30	인사부

2. 각 부서에 소속된 사원 조회

```
SELECT *
FROM EMP
WHERE DEPTNO = 10;
```

```
SELECT *
FROM EMP
WHERE DEPTNO = 20;
```

```
SELECT *
FROM EMP
WHERE DEPTNO = 30;
```

7.3 고급 JPA 기능



SQL 실행 횟수 = 1 + N

7.3 고급 JPA 기능

실제 JPA 코드 예

```
@Entity
@Table(name = "EMP")
public class Employee {

   @Id
   private Long empno;

   private String ename;

   @ManyToOne(fetch = FetchType.LAZY)
   @JoinColumn(name = "DEPTNO")
   private Department department;
}
```

```
@Entity
@Table(name = "DEPT")
public class Department {

   @Id
   private Long deptno;

   private String dname;

   @OneToMany(mappedBy = "department")
   private List<Employee> employees = new ArrayList<>();
}
```

```
@Service
@RequiredArgsConstructor
public class DepartmentService {

   private final DepartmentRepository departmentRepository;

   public void findDepartments() {

      List<Department> departments =
            departmentRepository.findAll();

      for (Department department : departments) {

         System.out.println(department.getDname());

         for (Employee employee : department.getEmployees()) {

            System.out.println(employee.getEname());
         }
      }
   }
}
```

1

N

7.3 고급 JPA 기능

연관관계의 일관성 (Consistency Management)

- 개념
  - 객체 간 양방향 관계에서는 두 참조가 항상 일관되도록 유지해야 됨
  - 한쪽만 설정하면 실제 관계 불일치 문제가 발생됨

- 예시

```
// 잘못된 코드 ( member만 team을 참조하고 team은 member 참조 안함 )
member.setTeam(team);


// 올바른 코드 ( member와 team이 서로 참조 )
member.setTeam(team);
team.getMembers().add(member);
```

7.3 고급 JPA 기능

연관관계의 일관성 (Consistency Management)

- 실무팁
  - 유틸리티 편의 메서드를 만들어 관계를 동시 유지함

```
public void addMember(Member member) {
   members.add(member);   // Team이 List<Member> 참조
   member.setTeam(this);     // Member가 Team 참조
}
```

```
public void removeMember(Member member ) {
      members.remove(member);
      member.setTeam(null);
   }
```

7.3 고급 JPA 기능

트랜잭션 내 변경 감지 (Dirty Checking)

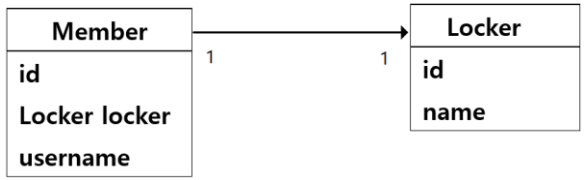
- 개념
  - 영속성 컨텍스트는 엔티티의 변경 여부를 추적하여 자동으로 UPDATE SQL를 실행함
  - 연관관계가 변경되면 외래키(FK) 업데이트도 자동 반영됨

- 예시

```
@Transactional
public void changeTeam(Long memberId, Team newTeam) {
   Member m = repository.findById(memberId);
   m.setTeam(newTeam);     // 외래키 변경 감지 & UPDATE SQL 자동 실행
}
```

7.3 고급 JPA 기능

일대일 단방향 예시




```
@Entity
@Table(name = "member")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Member {

    @Id
    private Long id;

    @Column(nullable = false, length = 100)
    private String username;

    // 1:1 단방향 (Member → Locker)
    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "locker_id", unique = true)
    private Locker locker;
}
```

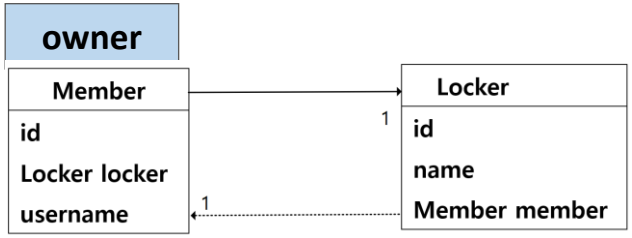
```
@Entity
@Table(name = "locker")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Locker {

    @Id
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;
}
```

7.3 고급 JPA 기능

일대일 양방향 예시




```
@Entity
@Table(name = "member")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Member {

    @Id
    private Long id;

    @Column(nullable = false, length = 100)
    private String username;

    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "locker_id", unique = true)
    private Locker locker;

    // 간단한 양방향 편의 메서드
    public void changeLocker(Locker locker) {
        this.locker = locker;
        locker.setMember(this);
    }
}
```

```
@Entity
@Table(name = "locker")
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
@Builder
public class Locker {

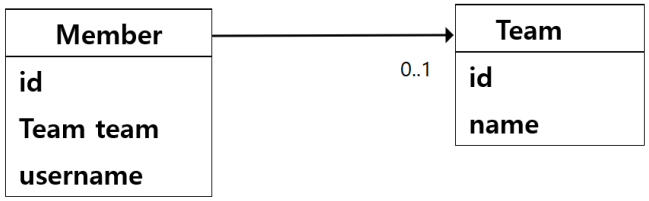
    @Id
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @OneToOne ( mappedBy = "locker")
    private Member member;
}
```

7.3 고급 JPA 기능

다대일 단방향 예시




```
@Entity
@Table(name = "member")
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
@Builder
public class Member {

   @Id
   @Column(name = "member_id")
   private Long memberId;

   @Column(nullable = false, length = 100)
   private String username;

   // 다대일 단방향 (Member → Team)
   @ManyToOne(fetch = FetchType.LAZY)
   @JoinColumn(name = "team_id")  // FK 컬럼명 지정
   private Team team;
}
```

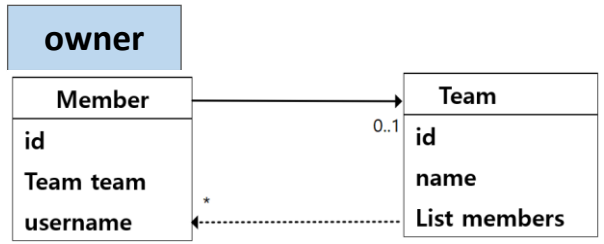
```
@Entity
@Table(name = "team")
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
@Builder
public class Team {

   @Id
   private Long id;

   @Column(nullable = false, length = 100)
   private String name;
}
```

7.3 고급 JPA 기능

다대일 양방향 예시




```
@Entity
@Table(name = "member")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Member {

    @Id
    private Long id;

    @Column(nullable = false, length = 100)
    private String username;

    // 주인: FK 컬럼 보유 (member.team_id → team.id)
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "team_id")
    private Team team;

    /** 간단한 양방향 편의 메서드 */
    public void changeTeam(Team team) {
        this.team = team;
        team.getMembers().add(this);
    }
}
```

```
@Entity
@Table(name = "team")
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
@Builder
public class Team {

    @Id
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

   // 양방향: 비주인(mappedBy = "team")
    @OneToMany(mappedBy = "team")
    @Builder.Default
    private List<Member> members = new ArrayList<>();  //초기화

}
```

7.3 고급 JPA 기능

Cascade (연쇄 저장/삭제)

- 개념
  - 부모(master) 엔티티의 영속성 상태 변화가 자식(slave) 엔티티에 전이(Cascade)되는 기능
- 내부 동작
  - CascadeType.PERSIST : 부모 저장 시 자식도 자동 저장
  - CascadeType.REMOVE : 부모 삭제 시 자식도 자동 삭제
  - CascadeType.ALL : PERSIST 와 REMOVE 한번에 적용
- 실무팁
  - 부모-자식 관계가 명확히 1:N 종속일때만 사용 ex. Order ----> OrderItem
  - 독립적으로 재사용되는 엔티티에는 사용 불가

7.3 고급 JPA 기능




```
Order order = new Order();
order.setOrderName("홍길동 주문");

// 자식 객체 생성
OrderItem item1 = new OrderItem();
item1.setProductName("노트북");

OrderItem item2 = new OrderItem();
item2.setProductName("마우스");

// 부모와 자식 연결
order.addItem(item1);
order.addItem(item2);

// 부모만 저장
em.persist(order);
```

```
// 부모 조회
Order order = em.find(Order.class, 1L);

// 부모만 삭제
em.remove(order);
```

7.3 고급 JPA 기능

orphanRemoval (고아객체 자동 삭제)

- 개념
  - 부모(master)와의 연관이 끊긴 자식 엔티티 자동 삭제 기능
  - 데이터 무결성 유지에 유용하지만 잘못 설정하면 데이터 손실 위험 가능
- 내부 동작
  - 부모 컬렉션에서 자식을 제거하면 DB에서도 자동으로 자식 삭제 DELETE SQL 실행
  - **Cascade 와 함께 사용하면 부모-자식 생명주기 일체화 가능**
- 실무팁
  - 부모가 존재하지 않으면 자식이 의미 없는 경우에만 사용
    ex. Order 삭제 시 OrderItem도 삭제되어야 하는 경우

7.3 고급 JPA 기능




```
Department dept = em.find(Department.class, 10);
Employee emp = dept.getEmployees().get(0);

dept.removeEmployee(emp);
```

7.3 고급 JPA 기능

Cascade vs orphanRemoval

항목	Cascade.ALL	orphanRemoval=true
목적	부모에서 자식으로 저장/삭제/수정 전파	부모 컬렉션에서 제거된 자식 자동 삭제
작동 시점	save(), deleteById()	관계 해제 ex. remove() + setTeam(null)
주요 효과	부모 저장/삭제 시 자식도 함께 저장/삭제	부모에서 제외된 자식은 DELETE 자동 실행
조합사용	함께 사용하면 완벽한 생명주기 관리 가능	

7.3 고급 JPA 기능

Cascade + orphanRemoval 예시

```
@Entity
@Table(name = "member")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
@ToString(exclude = "team")
public class Member {

    @Id
    private Long id;

    @Column(nullable = false, length = 100)
    private String username;

    // 주인: FK 컬럼 보유 (member.team_id → team.id)
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "team_id")
    private Team team;

    /** 간단한 양방향 편의 메서드 */
    public void changeTeam(Team team) {
        this.team = team;
        team.getMembers().add(this);
    }
}
```

```
@Entity
@Table(name = "team")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Team {

    @Id
    private Long id;

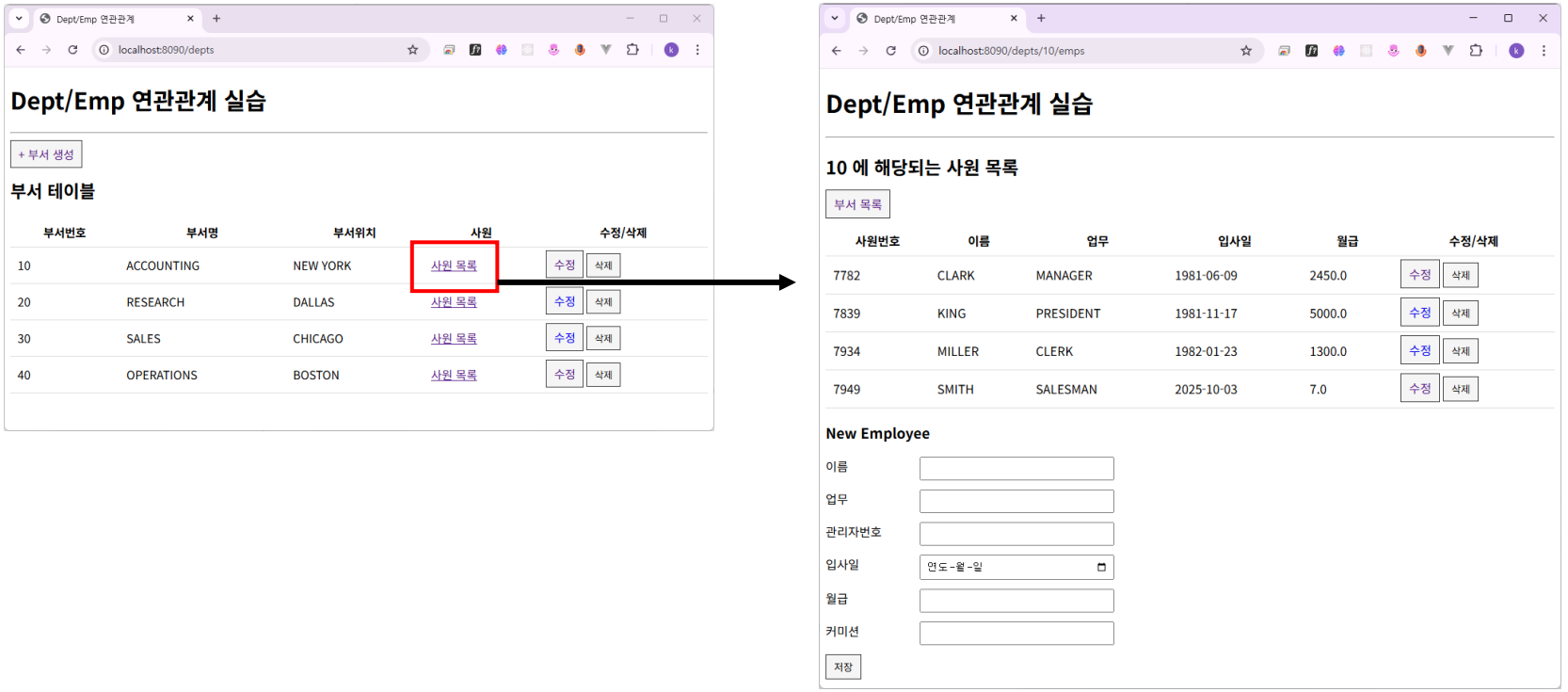
    @Column(nullable = false, length = 100)
    private String name;

   // 양방향: 비주인(mappedBy = "team")
@OneToMany(
        mappedBy = "team", fetch = FetchType.LAZY,
        cascade = CascadeType.ALL, orphanRemoval = true
    )    private List<Member> members = new ArrayList<>();

   /** 멤버 제거 시 자동 삭제 (orphanRemoval) */
    public void removeMember(Member member) {
        members.remove(member);
        member.setTeam(null);          // 관계 해제
    }
}
```

7.3 고급 JPA 기능

실습 (1:N 양방향 연관관계)



7.3 고급 JPA 기능

실습 (1:N 양방향 연관관계)

```
// Employee.java
package com.example.demo.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDate;

@Entity
@Table(name = "EMP")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Employee {

   @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
   @Column(name = "EMPNO")
   private Integer empno;

   @Column(name = "ENAME", length = 10)
   private String ename;

   @Column(name = "JOB", length = 9)
   private String job;

   @Column(name = "MGR")
   private Integer mgr;

   @Column(name = "HIREDATE")
   private LocalDate hiredate;

   @Column(name = "SAL")
   private Float sal;

   @Column(name = "COMM")
   private Integer comm;

   @ManyToOne(fetch = FetchType.LAZY, optional = false)
   @JoinColumn(name = "DEPTNO", nullable = false,
         foreignKey = @ForeignKey(name = "fk_emp_dept"))
   private Department dept;
}
```

7.3 고급 JPA 기능

실습 (1:N 양방향 연관관계)

```
// Department.java
package com.example.demo.entity;

import jakarta.persistence.*;
import lombok.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "DEPT")
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor @Builder
public class Department {

    @Id
    @Column(name = "DEPTNO", nullable = false)
    private Integer deptno;

    @Column(name = "DNAME", length = 14)
    private String dname;

    @Column(name = "LOC", length = 13)
    private String loc;

    // 1:N - cascade + orphanRemoval
    @OneToMany(mappedBy = "dept",
            cascade = CascadeType.ALL,
            orphanRemoval = true)
    @Builder.Default
    private List<Employee> employees = new ArrayList<>();

    // 연관관계 편의 메서드
    public void addEmployee(Employee e) {
        employees.add(e);
        e.setDept(this);
    }

    public void removeEmployee(Employee e) {
        employees.remove(e);
        e.setDept(null);
    }
}
```

7.3 고급 JPA 기능

실습 (1:N 양방향 연관관계)

```
@Override
  @Transactional
  public DepartmentDTO create(DepartmentDTO dto) {
    // 신규 부서 등록
    Department dept = toEntity(dto);
    dept.getEmployees().clear();      // 직원 리스트 초기화
    Department saved = departmentRepository.save(dept);
    return toDTO(saved);
  }

  @Override
  @Transactional
  public DepartmentDTO update(Integer deptNo, DepartmentDTO dto) {
    // 부서 수정 (존재하지 않으면 예외)
    Department dept = departmentRepository.findById(deptNo)
        .orElseThrow(() -> new EntityNotFoundException("Department not found: " + deptNo));
    dept.setDname(dto.getDname());
    dept.setLoc(dto.getLoc());
    // 영속성 컨텍스트가 변경 감지(Dirty Checking)로 자동 update 처리
    return toDTO(dept);
  }
```

```
@Override
  @Transactional
  public void delete(Integer deptNo) {
    // 부서 삭제
    // CascadeType.ALL + orphanRemoval 때문에, 소속 직원도 자동 삭제됨
    departmentRepository.deleteById(deptNo);
  }
```

7.4 JPQL

JPQL (Jakarta Persistence Query Language)

- 개념

  JPA에서 사용하는 객체 지향 쿼리 언어.

  DB 테이블을 직접 대상으로 SQL을 작성하는 것이 아니라, JPA의 엔티티(Entity)객체와 객체의 속성(field)을 대상으로 조회하는 언어.

- 공식 사이트
  - **https://docs.jboss.org/hibernate/orm/6.4/querylanguage/html_single/Hibernate_Query_Language.html#basic-concepts**

7.4 JPQL

SQL vs JPQL

구분	SQL	JPQL
조회 대상	테이블	**엔티티**
컬럼 참조	컬럼명	**엔티티 속성명**
테이블병	employee	**Employee**
별칭	선택적	**엔티티 별칭 사용**
실행 대상	DB	**JPA가 SQL로 변환 후 DB 실행**

```
SELECT *
FROM employee;
```

```
SELECT e
FROM Employee  e
```

```
SELECT ename, sal
FROM employee
        WHERE sal >= 3000;
```

```
SELECT e.ename, e.sal
FROM Employee e
WHERE  e.sal >= 3000;
```

7.4 JPQL

JPQL 실행 과정



7.4 JPQL

식별자 및 대소문자 구별

`select`, `SeLeCT`, `sELEct`, and `SELECT`	All the same, `select` is a keyword
`upper(name)` and `UPPER(name)`	Same, `upper` is a function name
`from BackPack` and `from Backpack`	Different, refer to different Java classes
`person.nickName` and `person.nickname`	Different, since the path expression element `nickName` refers to an attribute of an entity defined in Java
`person.nickName`, `Person.nickName`, and `PERSON.nickName`	All different, since the first element of a path expression is an identification variable

7.4 JPQL

연산자

Precedence class	Type	Operators
Grouping and tuple instantiation		`( … )` and `(x, y, z)`
Case lists		`case … end`
Unary numeric	Unary prefix	`+`, `-`
Duration conversions	Unary postfix	`by day` and friends
Binary multiplicative	Binary infix	`*`, `/`, `%`
Binary additive	Binary infix	`+`, `-`
Concatenation	Binary infix	`||`
Nullness, emptiness, truth	Unary postfix	`is null`, `is empty`, `is true`, `is false`
Containment	Binary infix	`in`, `not in`
Between	Ternary infix	`between`, `not between`
Pattern matching	Binary infix	`like`, `ilike`, `not like`, `not ilike`
Comparison operators	Binary infix	`=`, `<>`, `<`, `>`, `<=`, `>=`
Logical negation	Unary prefix	`not`
Logical conjunction	Binary infix	`and`
Logical disjunction	Binary infix	`or`

7.4 JPQL

파라미터 바인딩

Parameter type	Examples	Usage from Java
Named parameters	`:name`, `:title`, `:id`	`query.setParameter("name", name)`
Ordinal parameters	`?1`, `?2`, `?3`	`query.setParameter(1, name)`
JDBC-style parameters 💀 deprecated	`?`	`query.setParameter(1, name)`

```
@Query(value = "SELECT c FROM Study as c WHERE c.id = :x")
Study findTodoById(Long x);

@Query(value = "SELECT c FROM Study as c WHERE c.description  LIKE ?1 ")
List<Study> findTodoByDescriptionLike(String description);
```

7.4 JPQL

단일함수 및 그룹함수

Aggregate function	Argument type	Result type	JPA standard / ANSI SQL standard
`count()`, including `count(distinct)`, `count(all)`, and `count(*)`	Any	`Long`	✓/✓
`avg()`	Any numeric type	`Double`	✓/✓
`min()`	Any numeric type, or string	Same as the argument type	✓/✓
`max()`	Any numeric type, or string	Same as the argument type	✓/✓
`sum()`	Any numeric type	See table below	✓/✓

7.4 JPQL

조인 (join)

```
SELECT d, e
FROM Department d
INNER JOIN d.employees e

SELECT d, e
FROM Department d
LEFT JOIN d.employees e


SELECT e.ename, d.dname
FROM Employee e
JOIN Department d
   ON e.dept.deptno = d.deptno


SELECT d
FROM Department d
JOIN FETCH d.employees     # N+1 해결
```

7.4 JPQL

Spring Data JPA + JPQL 사용

```
public interface StudyRepository extends JpaRepository<Study, Long>{

 @Query(value = "SELECT c FROM Study as c")
 List<Study> findTodoList();

 @Query(value = "SELECT c FROM Study as c WHERE c.id = :x")
 Study findTodoById(Long x);

 @Query(value = "SELECT c FROM Study as c WHERE c.description  LIKE ?1 ")
 List<Study> findTodoByDescriptionLike(String description);

 @Query(value = "SELECT c.description FROM Study as c")
 List<String> findTodoDescriptionList();

 @Query(value = "SELECT c.id as xxx , c.description as yyy FROM Study as c")
 List<Map> findTodoIdAndDescriptionList();
}
```

수고하셨습니다.